
KillerPDF

Technical Specification & Complete Feature Brochure

VERSION 1.6.4

A native Windows PDF viewer, annotator and light editor.
Single self-contained executable. Built on WPF and PDFium.

*This document is itself a KillerPDF showcase - open it in the app
and explore the outline, the forms, and the annotation pages.*

KillerPDF.net

.NET Framework 4.8 · x64 · Windows 10/11 · GPL-3.0

INTRODUCTION

About the Maker

Steve the Killer · field technician, systems administrator, builder of sharp tools

KillerPDF was born out of a ticket queue. I spend my days as a field tech and systems administrator at a large MSP, and a steady share of that work is always the same shape: a user can't fill a PDF, a form won't flatten, a license won't activate, a viewer throws an error on a file that should just open. Adobe Acrobat tickets, install it, repair it, license it, work around it, added up to hours every week that never felt like they were actually fixing anything.

So I built the tool I wished my users had: one fast, self-contained executable that opens the broken files, fills forms, signs documents, OCRs scans and gets out of the way. No installer, no account, no subscription. It is deliberately overbuilt for the encrypted, broken-xref and rotated-and-clipped PDFs that make other viewers give up, because those are the files that generate tickets. Everything in this brochure is real and shipping.

"I got tired of closing the same Adobe ticket. So I wrote a PDF app that just works, ships as one .exe, and never asks the user to log in. That app is KillerPDF."

KillerPDF is free software (GPL-3.0). Find it, and everything else I build, at [KillerPDF.net](https://killerpdf.net).

About this Brochure

The complete technical specification and feature list for KillerPDF 1.6.4.

This document is the complete technical specification and feature list for KillerPDF version 1.6.4, and a working showcase of the app itself. Open it inside KillerPDF and it becomes a tour: the outline is a deep nested bookmark tree (now editable in the sidebar), Part IV has a real interactive AcroForm and a printed-style form to mark up, and the annotation pages are seeded for screenshots. Every tool, theme, shortcut and constant here is taken directly from the source.

How this document is organised

- › Part I, the platform: tech stack, architecture, render pipelines, view modes and the calculation deep-dive.
- › Part II, every tool, in the GUI and under the hood, including the new command-line interface.
- › Part III, reference: keyboard map, themes, languages, robustness, standards conformance, the hard numbers, and a plain-language glossary.
- › Part IV, a live showcase you can interact with in the viewer.

Any page here makes a clean example document for marketing captures.

CONTENTS

Table of Contents

Part I, Platform & Architecture

Product & tech stack · Architecture overview · View modes · Rendering & zoom mathematics · Interaction mathematics

Part II, Tools & Features

Text, annotation, ink & images · Crop, links & forms · Stamp, search, OCR & signing · Print, rotate, pages & import · Tabs, outline & thumbnails · Bookmarks, redo & jump history · Zoom & undo · Command line

Part III, Reference

Keyboard shortcuts & tool keys · Themes & accents · Localization · File handling & robustness · Standards conformance · Notable engineering details · Specifications & constants · Glossary

Part IV, Live Showcase

Interactive feedback form · Sample application form · Annotation playground



Platform & Architecture

- › The technology and ideas the app is built on
- › Single partial-class WPF architecture, two render pipelines
- › A zoom-stable coordinate space shared by every view

Product & Tech Stack

A native Windows PDF viewer, annotator and light editor shipped as one self-contained executable.

It renders with PDFium (via Docnet.Core), reads text with PdfPig and writes the PDF object model with PdfSharpCore, vendored into the repo and patched for standards conformance. The whole UI is one partial class, MainWindow, spread across 38 files, about 22,000 lines of C# (roughly 29,000 app-wide).

Component	Detail
Version	1.6.4 (mirrors the csproj Version)
Target framework	.NET Framework 4.8 (net48), WinExe, x64
UI toolkit	WPF, custom chrome with native resize and snap
Distribution	Single .exe; Costura embeds DLLs, native OCR libs self-extract
Render engine	PDFium via Docnet.Core 2.6
Text engine	PdfPig 0.1.14, search, selection, font sniffing
Write engine	PdfSharpCore 1.3.67, vendored under third_party/ with six conformance patches
Signing engine	PDFsharp 6.2 (separate namespace), CMS / SHA-256
Image codecs	SixLabors.ImageSharp 2.1.13, import, clipboard paste, signatures
OCR	Tesseract 5.2, native libs embedded, packs on demand
Command line	Full headless CLI: merge, split, decrypt, rasterize, flatten, print, OCR, batch-resave

- › Four view modes sharing one coordinate space; a full annotation suite (text, highlight, ink, lines, images, signatures, covers).
- › Editing: crop, rotate, transform, page reorder / insert / delete, bookmark editing, stamp, watermark, page numbering.
- › Interactive AcroForm filling, link handling, full-text search, jump history, OCR and cryptographic signing.

Architecture Overview

One class, many files.

Every part of the window shares the private fields declared in `MainWindow.xaml.cs`, while concerns are split across files such as `Viewport.cs`, `Annotations.cs`, `Tabs.cs` and `Zoom.cs` using C#'s partial classes. Dialogs are standalone classes; cross-cutting work lives under `Services/`.

Two render pipelines

Single, Two-Page and Grid render into the primary tile (`PageImage + _annotationCanvas`) through `RenderPage(idx)`, with extra tiles built by `RenderAdditionalPages`. Continuous mode owns a separate path: `SetupContinuousView` builds one overlay per page and `RenderContinuousPages` streams bitmaps on a background thread. Since 1.6.3 the continuous strip is virtualized: only a window of pages around the viewport holds real bitmaps, so a 243-page image-heavy PDF costs a few hundred MB instead of gigabytes.

INVARIANT: `RenderPage` is Single / Grid / TwoPage only and no-ops in Continuous. It calls `ClearSecondaryPages`, which would otherwise repoint every overlay at the hidden primary tile and paint annotations off-screen.

Per-tab sessions & the LRU cache

Each open document is a `DocumentSession` holding its zoom, fit and view mode, grid column count, tool, page, scroll offsets and the per-document dictionaries (annotations, render dims, rotations, form values, undo and redo stacks, jump history). Switching tabs re-points the live fields at the session by reference. Each session also carries a frozen-bitmap `RenderCache` keyed by (page, size-bucket, rotation), so returning to a recent tab skips PDFium entirely; whole-tab caches evict beyond the three most recent.

View Modes

Four view modes, switchable from the toolbar or with F5 to F8.

All four place annotations against the same 2048-longest-side coordinate space, so marks never drift when you change how pages are laid out.

Mode	Behaviour
Continuous (F5)	Vertical scroll of every page; overlays built up front, bitmaps streamed asynchronously with cancellation and released once far off-screen. The default mode. The current page follows the viewport; clicks never yank the view.
Single (F6)	One page in the primary tile; fit and zoom math use the selected page's render dimensions.
Two-Page (F7)	Primary tile plus one secondary tile side by side, paired as spreads (0,1)(2,3)... Arrow keys and PgUp / PgDn move one spread at a time.
Grid (F8)	A thumbnail overview of the whole document; the column count is the single source of truth for grid zoom and is stored per tab. The statusbar counter follows the tile nearest the viewport center.

Single, Two-Page and Grid share the primary panel; Continuous uses its own stacked panel. Secondary tiles key the render cache by their rounded primary-tile width, so zooming reuses cached bitmaps.

How It's Calculated: Rendering & Zoom

Two numbers describe every page: where things are (zoom-independent) and how many pixels we draw (zoom-dependent).

Keeping them separate is what makes annotations stay put while text stays crisp.

1. The render-dim space

Every page is normalised so its longest side is exactly 2048 units; annotation coordinates are stored here, identical at every zoom and across all four pipelines.

```
maxDim = max(pageWidthPt, pageHeightPt)
rdW = round(2048 * pageWidthPt / maxDim)
rdH = round(2048 * pageHeightPt / maxDim) // longest side -> 2048
```

2. Decoupled bitmap resolution

The bitmap rasterises at a resolution that follows DPI and zoom, so a 3x page is drawn from 3x the pixels rather than upsampled. The 6144 px ceiling bounds memory.

```
scaledMax = min( 6144, int( 2048 * max(dpiScaleX, dpiScaleY) * max(1.0, zoom) ) )
```

3. The coordinate Y-flip

PDF uses a bottom-left origin in points; WPF canvases a top-left origin in DIP. Every mark crosses that boundary.

```
sx = renderW / pdfW      sy = renderH / pdfH
canvasX = left * sx
canvasY = renderH - top * sy // flip the vertical axis
```

How It's Calculated: Interaction

The same precision applies to what you do with the mouse.

Cursor-anchored zoom

Ctrl+wheel zooms around the cursor by a constant 10% ratio per notch: the view scales instantly while the wheel is moving and the crisp high-resolution re-render happens once when the wheel rests. The cursor position and scroll offsets are captured before the zoom changes, then the offsets that keep that point fixed are applied once layout settles.

```
ratio    = newZoom / oldZoom
newHOfff = (oldHOfff + cursorX) * ratio - cursorX
newVOfff = (oldVOfff + cursorY) * ratio - cursorY    // clamped at >= 0
```

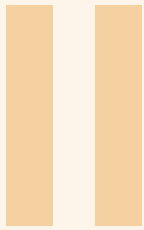
Grid zoom by column count

Grid snaps to a whole number of columns: the count is authoritative and the zoom is derived from it, so the grid lays out exactly N pages with no leftover gap.

```
GridZoomForN(n) = (viewportW - 24) / ( n * (rdW + 12) )
```

Cache keys & eviction

```
key = ( pageIndex, sizeBucket, rotation )
sizeBucket = scaledMax (primary) or round(primaryDipW) (tiles)
evict : keep the 3 most-recently-used tabs; clear the rest
```



Tools & Features

- › Every annotation, editing and document tool
- › How each behaves in the GUI
- › What happens under the hood, with real names

Text & Typewriter

Drop a text box anywhere, or re-edit existing PDF text in place.

Text [T / 2]

IN THE GUI. Click to place a box and type (Enter saves, Escape cancels); double-click existing text to re-edit. A floating bar sets color, opacity, fill, point size, font and Bold / Italic / Strike / Underline.

UNDER THE HOOD. PlaceTextBox builds a flat WPF TextBox with live resize handles; CommitActiveTextBox converts it to a TextAnnotation. Re-editing drops a page-color CoverAnnotation (paired by PairId) over the original; font and size are sniffed from PdfPig with a mojibake guard, falling back to Segoe UI 14pt.

Highlight, Strikethrough & Underline

Three drag-to-mark styles sharing one annotation type, with an optional eraser.

Highlight / Strike / Underline [H / 4]

IN THE GUI. Drag a rectangle to mark text. Highlight fills; strikethrough and underline draw a thin band. The eraser works two ways: a Brush carves smooth, round-edged transparent gaps out of a fill, while Box drags a red marching-ants marquee that deletes whole annotations inside it.

UNDER THE HOOD. All three are a HighlightAnnotation tagged with a HighlightStyle and RGBA color; DrawRect returns full bounds or a band. HighlightEraseGeometry subtracts the union of round-cap eraser strokes as one anti-aliased geometry, preserved through save.

Ink (Draw) & Line

Freehand ink and straight lines, both backed by one ink annotation type.

Draw & Line [D / 5 · L / 3]

IN THE GUI. Draw paints freehand strokes and keeps drawing without re-arming; Line drags a straight segment with an optional Level snap. The bar offers color, opacity, width and an eraser (Draw) or level (Line).

UNDER THE HOOD. Both create an InkAnnotation (points + stroke width + RGBA, default red 2.0px). Draw accumulates points on move; Line keeps two points. A minimum-length guard rejects click jitter.

Image & Cover

Place a picture on the page; covers provide whiteout for text replacement.

Image [1 / 6]

IN THE GUI. Pick Image, click and choose a file (or just paste with Ctrl+V); it lands at 50% scale, corner-resizable. Covers are the opaque whiteout half of an in-place text re-edit.

UNDER THE HOOD. ImageAnnotation stores Base64 bytes with the decoded bitmap cached, drawn at SourceWidth*Scale. Decoding runs through SixLabors.ImageSharp 2.1.13, and images with broken DPI metadata (WhatsApp photos, some scans) are kept within Adobe's supported 3-14,400 point page range. CoverAnnotation derives from HighlightAnnotation with forced-opaque alpha, paired by PairId and flattened solid on export.

Crop

Trim pages with numeric precision, on one page or all.

Crop [C / 8]

IN THE GUI. A default box inset 8% from the edges plus a bar with X / Y / W / H inputs, a unit selector (points, inches or %), current- or all-pages scope, Remove-crop and Apply / Cancel.

UNDER THE HOOD. ApplyCrop converts the canvas rect to PDF coordinates (rotation-aware via CanvasToPdfRect), clamps to the media box, writes /CropBox and /TrimBox with PdfSharpCore, then saves and reloads. Every save also strips degenerate zero-size crop boxes, healing files damaged by older builds.

Links

Existing PDF links become clickable; you can copy or remove them.

Links

IN THE GUI. Links become hand-cursor overlays: left-click follows an internal jump or external URI; right-click offers Copy URL or Remove Link. Internal jumps are recorded in the jump history, so Alt+Left retraces them.

UNDER THE HOOD. RenderPageLinks reads each page's /Annots, parses /Link /Rect and the /A action and builds transparent overlays; object-stream PDFs fall back to PDFium link enumeration through a cached document handle that is released before every save. On save, StripLinkAnnotationBorders removes borders so they don't strobe elsewhere.

Forms (AcroForm)

Fill interactive text fields, checkboxes, radios and dropdowns.

Forms

IN THE GUI. Widget fields overlay the page with a cream background and accent-on-focus border; text fields show a font-size stepper. Checkboxes, radios and dropdowns are editable and written back on save.

UNDER THE HOOD. Widgets parse into FormFieldInfo (type /Tx, /Btn or /Ch; /Rect with full 0/90/180/270 rotation handling; /DA font size). Values persist by object number / field name and are written by WriteFormValuesToDocument, which also regenerates each field's appearance and sets /NeedAppearances so other PDF readers redraw correctly. Try the Interactive Feedback Form in Part IV.

Stamp, Watermark & Page Numbers

Two independent overlays: page numbers and a text or image watermark.

Stamp / Watermark [S / 0]

IN THE GUI. A live-preview editor with a page stepper. Page numbers take a start value, a format ({n} current, {N} total), size, color, a range like 1-3,5, eight positions plus custom and a spread mirror. Watermarks take text or an image, size / color or scale, opacity, angle and position.

UNDER THE HOOD. A StampSpec holds every setting (watermark default DRAFT 64pt 25% at 45 degrees). DrawStampsOnDocument flattens via PdfSharpCore XGraphics: watermark under, numbers on top, image opacity baked through a color matrix, rotation via translate-plus-rotate.

Search

Debounced full-text search with persistent per-page highlights.

Search [Ctrl+F · F3]

IN THE GUI. Ctrl+F toggles a draggable search bar (250ms debounce); the status shows current / total, and F3 / Shift+F3 step to the next / previous match from anywhere in the app (F3 opens search when it isn't showing). Enter, Shift+Enter and the wheel walk the matches too.

UNDER THE HOOD. SearchService.Search scans each page's PdfPig GetWords() case-insensitively (single- and multi-word) for per-page rectangles. ApplySearchHighlights repaints them on every render: the current match a brighter accent outline, the rest dim.

OCR

On-device text recognition in nine languages, with a searchable-PDF text layer.

OCR [Ctrl+Shift+O · Ctrl+Shift+I]

IN THE GUI. Ctrl+Shift+O OCRs the page to the clipboard; Ctrl+Shift+I drags a region; a language menu manages models and Escape cancels. You can also write the recognised text back as an invisible, transparent layer over the image, making a scan fully selectable and searchable without changing how it looks.

UNDER THE HOOD. OcrNativeBootstrap self-extracts the embedded x64 Tesseract / Leptonica libraries to a per-version cache. Nine languages (matching the interface locales, Japanese included) download on demand into a version-independent tessdata folder, in two tiers: a standard model (~4 MB per language) and a higher-accuracy model (~14 MB). Pages render at 2600px (~300 DPI) and feed OcrService.RecognizeBgra, returning text and a mean confidence.

Digital Signing & Visual Signatures

Certificate-based CMS signatures with whole-chain SHA-256, plus reusable drawn signatures.

Digital Signing (cryptographic)

IN THE GUI. Pick a certificate from a .pfx / .p12 file or the Windows store, add an optional reason, location and contact, and write a signed copy. Visual signatures (G / 7) are separate: hand-drawn or imported images saved for reuse.

UNDER THE HOOD. Built on PDFsharp 6.2. WindowsCertificateStore enumerates the personal store; KillerCmsSigner implements IDigitalSigner on .NET SignedCms (CNG-capable, so token / cloud keys can prompt for a PIN), with SHA-256, the whole chain and a 16,384-byte /Contents reservation. SignatureStore persists drawn signatures as JSON in %LOCALAPPDATA%. Because any later edit invalidates a signature's digest, resaving a signed file strips the dead signature value rather than shipping one that fails strict validation.

Print

A real print-preview with N-up, scaling, margins and alignment.

Print [Ctrl+P]

IN THE GUI. Preview with N-up (1 / 2 / 4 / 6 / 9), per-page scale (fit, 100% or custom), alignment, margins, orientation, page navigation and a rendering progress label. Copies and custom scale are numeric fields with spinners; the dialog remembers the last printer, orientation, color and two-sided choice.

UNDER THE HOOD. Built on System.Printing and ReachFramework. On Print, the selected pages re-rasterize at 300 DPI from a flattened copy (annotations baked in), compose into a FixedDocument and go to the spooler via PrintDialog.

Rotate & Transform

Quick 90-degree turns or a full transform with scale and flip.

Rotate / Transform [R / 9]

IN THE GUI. Quick rotations are on the context menu and the R key; the Transform window adds angle, scale, flip, straighten-by-line and fixed- versus resize-page modes at full resolution, with a live preview.

UNDER THE HOOD. Structural rotation routes through SaveTempAndReload, which zeroes /Rotate before saving (PDFium sizes from the unrotated MediaBox, so rotated content would clip) and restores it in memory. ApplyPageTransform burns annotations in, composes flip / scale / rotate-with-expand and re-encodes.

Page Operations & Import

Reorder, insert, delete, duplicate, extract and split pages; import images, folders and archives.

Page operations & import

IN THE GUI. Rotate, delete, insert or append blank A4 pages, move up / down, duplicate, extract or split; dragging a sidebar thumbnail reorders pages. Open images (JPG, PNG, BMP, GIF, TIFF) as pages; folders recurse and .zip extracts to temp, with a choice to merge into one PDF or open each in its own tab.

UNDER THE HOOD. Structural edits route through SaveTempAndReload(keepAnnotations:true), which rebuilds the rotation and render-dim maps and refreshes thumbnails on a background thread. Pixel-to-point conversion uses each image's own DPI (96 fallback); a 50-file cap (MaxDropFiles) prompts and truncates.

Tabs, Outline & Thumbnails

Multi-document tabs with flicker-free rendering, plus an editable bookmark tree and lazy thumbnails.

Tabs [Ctrl+Tab · Ctrl+W] · Sidebar [Ctrl+B]

IN THE GUI. Open many documents as tabs; the active tab stays visible with an overflow menu. A dockable sidebar has a Pages (thumbnail) tab and an Outline tab that mirrors the PDF's bookmark tree (this document has a deep one, try it here) and edits it in place: add via the row at the top, rename inline (F2), nest, reorder, retarget and delete, with Ctrl / Shift multi-select and full undo.

UNDER THE HOOD. Backed by DocumentSession. The strip is reconciled in place via a session-to-element cache, so no flicker on switch, drag or resize; lazy tabs defer loading until activated. LoadOutlines walks `_doc.Outlines` into a TreeView; titles from password-protected files are re-decoded to dodge mojibake, and thumbnails lazy-load on a concurrency-limited thread at ~288px.

Redo & Jump History

Browser-style history for both your edits and your movements through the document.

Redo [Ctrl+Y] · History [Alt+Left / Alt+Right]

IN THE GUI. Ctrl+Y (or Ctrl+Shift+Z) re-applies undone actions: annotations, text edits, stamps, clears and document-level operations alike. Alt+Left / Alt+Right, or the mouse back / forward buttons, retrace bookmark, link, jump-box and Home / End jumps exactly like a browser.

UNDER THE HOOD. Redo entries mirror the `_undoStack` records; any new edit clears the redo chain, and both stacks live per tab in the `DocumentSession`. Jump history is a pair of page stacks (back / forward) fed by `RecordNavJump` at every hard jump; retracing pushes the current page onto the opposite stack, so the trail never dead-ends.

Zoom & Undo

Cursor-anchored zoom from 5% to 500%, and one undo history across annotations, pages and whole-document edits.

Zoom [Ctrl + / Ctrl - · Ctrl+0 · Ctrl+1/2/3] · Undo [Ctrl+Z]

IN THE GUI. Ctrl+wheel zooms around the cursor; in Grid the wheel steps the column count. An editable zoom box offers fit-width and fit-page, and Ctrl+1 / Ctrl+2 / Ctrl+3 set actual size / fit width / fit page from the keyboard. Ctrl+Z reverses the last change; auto-repeat is ignored so one keypress is one undo.

UNDER THE HOOD. ZoomMin 0.05, ZoomMax 5.0, ZoomStep 0.15; FitMode is None / Width / Page. _undoStack holds UndoEntry records tagged by UndoKind: single or grouped annotation, clear-page, stamp batch, bookmark edit, page snapshot or a full document byte snapshot.

Command Line

Every core operation also runs headless from a terminal, new in 1.6.4.

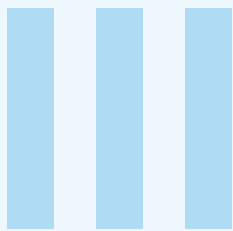
No window, meaningful exit codes (0 success, 1 failed, 2 bad usage), and it works even while the app is open. Each command reuses the exact pipeline its GUI equivalent runs: merges rewrite named-destination links, saves scrub the same structural hazards, decryption is lossless through PDFium, and OCR languages download on first use.

```
KillerPDF.exe --merge out.pdf a.pdf b.pdf scan.jpg
KillerPDF.exe --extract-pages in.pdf 1-3,5 out.pdf
KillerPDF.exe --split in.pdf pages\
KillerPDF.exe --decrypt locked.pdf open.pdf [--password p]
KillerPDF.exe --to-image in.pdf imgs\ --dpi 300 --format jpg
KillerPDF.exe --flatten in.pdf flat.pdf
KillerPDF.exe --print in.pdf --printer "HP LaserJet" --pages 1-4 --copies 2
KillerPDF.exe --ocr scan.pdf searchable.pdf --lang eng
KillerPDF.exe --batch-resave inDir\ outDir\ --log report.csv
KillerPDF.exe --help
```

Batch mode & the validation harness

--batch-resave resaves a single PDF or a whole folder tree headlessly through the standard open/save pipeline with per-file OK / SKIP / FAIL reporting. It exists so the standards-conformance harness (validation/, see Part III) can push thousands of corpus files through a real KillerPDF save and prove the output validates exactly as well as the input.

Rasterizing is rotation-safe at 150 or 300 DPI, printing drives the same 300 DPI flattened pipeline as the dialog, and --version / --help print and exit cleanly for scripting.



Reference

- › The complete keyboard map and tool keys
- › Six themes and the accent system
- › Nine languages, robustness, standards conformance and a glossary

Keyboard Shortcuts

KillerPDF is fully keyboard-driven; global shortcuts pause only while you are typing in a text box or form field. Press F1 for the in-app overlay, which now includes a visual keyboard view.

Key	Action
Ctrl+O / Ctrl+N	Open / New document
Ctrl+S / Ctrl+Shift+S	Save in place / Save As
Ctrl+P	Print
Ctrl+D / F4	Document info
Ctrl+, / F9	Settings
F12	About dialog
Ctrl+W / Ctrl+Q	Close tab / Close all tabs
Ctrl+Tab / Ctrl+Shift+Tab	Next / previous tab
Ctrl+F	Toggle search
F3 / Shift+F3	Next / previous search match (opens search when closed)
Ctrl+Z	Undo
Ctrl+Y / Ctrl+Shift+Z	Redo
Ctrl+C / Ctrl+V / Ctrl+A	Copy / paste / select all (annotations or text)
Ctrl+Shift+O / Ctrl+Shift+I	OCR current page to clipboard / begin OCR region
Ctrl+B	Collapse / restore sidebar
Ctrl + / Ctrl -	Zoom in / out (Grid: step columns)
Ctrl+0 / Ctrl+1 / Ctrl+2 / Ctrl+3	Reset zoom / actual size / fit width / fit page
F1 / Ctrl+?	Shortcuts overlay, LIST or KEYBOARD view
F2	Rename the selected bookmark
F5 / F6 / F7 / F8	Continuous / Single / Two-Page / Grid
F11	Full screen
Home / End	First / last page
PgUp / PgDn	Previous / next page (Two-Page: previous / next spread)
Alt+Left / Alt+Right	Back / forward through the jump history
Menu key / Shift+F10	Context menu at the current selection
Delete	Delete selected annotation(s) or bookmark
Arrow keys / Enter	Scroll / flip pages / confirm dialogs
Space (hold) / Escape	Pan cursor / Cancel or close overlay

Tool Keys

Every tool is reachable by its toolbar position (digits 1 to 9, 0) and a legacy letter; tooltips append the number, e.g. Highlight (4).

Key	Tool
V / 1	Select
T / 2	Text
L · U / 3	Line
H / 4	Highlight
D / 5	Draw (ink)
I / 6	Image
G / 7	Signature (opens picker)
C / 8	Crop
R / 9	Rotate (Transform window)
S / 0	Stamp

Themes & Accents

Six built-in themes, each a film-grain-textured palette. Three base themes also accept an accent overlay.

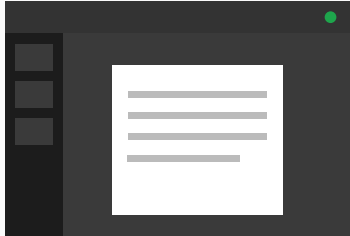
Theme	Description
Dark	The default, charcoal with brand green (#1EA54C)
Light	Light surfaces, deep green accent (#18843D)
Black	High-contrast, neon-green chrome (#00FF66)
Blood	Deep oxblood with a cream accent (#F8C99E)
Greed	Forest green with gold (#E0D49A)
Cyanotic	Deep blue with a warm accent (#E0D49A)

Accent variants

The Dark, Light and Black themes each accept one of six accent overlays: Green #1EA54C, Red #DD504B, Orange #E8962C, Teal #1FB8A8, Blue #50AEE8, Purple #B982E3. Each theme remembers its accent independently, in the app and on KillerPDF.net alike.

Theme Gallery

Every surface color below is read directly from the theme dictionaries in the source (Themes/*.xaml).



Dark

The default. Graphite chrome with the KillerTools green.

chrome	sidebar	panel	canvas	modal	text	accent
#333333	#1C1C1C	#3A3A3A	#3A3A3A	#1E1E1E	#E0E0E0	#1EA54C

Accepts all six accent overlays:

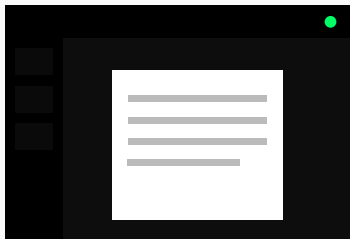


Light

Paper-bright chrome with a forest-green accent.

chrome	sidebar	panel	canvas	modal	text	accent
#F0F0F0	#DCDCDC	#E8E8E8	#B8B8B8	#FFFFFF	#1A1A1A	#1B5E20

Accepts all six accent overlays:



Black

True black, built for OLED, with a neon green accent.

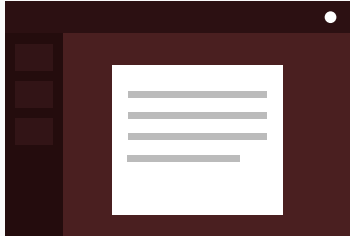
chrome	sidebar	panel	canvas	modal	text	accent
#000000	#000000	#0A0A0A	#0D0D0D	#000000	#FFFFFF	#00FF66

Accepts all six accent overlays:



Theme Gallery, continued

The three tinted themes recolor every surface and pin the accent to white.

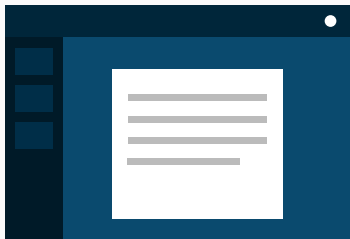


Blood

Deep crimson tint over every surface; fixed white accent.

chrome	sidebar	panel	canvas	modal	text	accent
						
#2C1012	#240C0D	#321416	#4A1F20	#1E0A0B	#FFFDE8	#FFFFFF

Fixed accent: white on the tinted canvas, selections as translucent white.

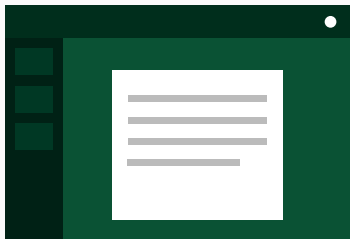


Cyanotic

Midnight blueprint blue; fixed white accent.

chrome	sidebar	panel	canvas	modal	text	accent
						
#00263A	#001A28	#002E48	#0A4A6E	#001624	#FFFDE8	#FFFFFF

Fixed accent: white on the tinted canvas, selections as translucent white.



Greed

Money-green felt; fixed white accent.

chrome	sidebar	panel	canvas	modal	text	accent
						
#002E1C	#002115	#003824	#0A5234	#001E13	#FFFDE8	#FFFFFF

Fixed accent: white on the tinted canvas, selections as translucent white.

Localization

KillerPDF ships nine UI languages.

Strings are per-locale resource dictionaries keyed `Str_*`, with English always present as the base so any key a translation omits falls back automatically. Most UI binds the resources dynamically, so a language switch is instant.

Locale	Name
English (en-US)	Base locale, ~500 string keys
Spanish (es)	Espanol
German (de-DE)	Deutsch
French (fr-FR)	Francais
Turkish (tr-TR)	Turkce
Bengali (bn)	Bangla
Japanese (ja-JP)	Nihongo, new in 1.6.2
Chinese Simplified (zh-CN)	Simplified
Chinese Traditional (zh-TW)	Traditional

`LocaleManager`.Apply hot-swaps the merged dictionaries to theme + en-US + the chosen locale; code-built surfaces (context menu, toolbar captions, annotate bars, the shortcuts overlay) are rebuilt on a change. Contributions come in as pull requests against `Strings/`, guided by `Strings/TRANSLATING.md`; every locale ships at full key parity with English.

File Handling & Robustness

KillerPDF is built to open the PDFs other viewers choke on, and to never lose your work to a malformed file.

Resilient open

- › Network / UNC paths are copied locally first to avoid 9P short-read corruption; owner-restricted /Encrypt PDFs prompt for a password.
- › An exception-driven fallback chain: xref corruption falls back to read-only then a PDFium repair; FlateDecode / EOF errors strip encryption and re-save losslessly; a zero-page page tree is guarded instead of crashing; anything else offers a repair.

Rotation-safe temp reload

SaveTempAndReload zeroes each page's /Rotate before saving (PDFium sizes from the unrotated MediaBox, so rotated content would clip) and restores it in memory; on xref failure it falls back to a PDFium save with FPDF_REMOVE_SECURITY or a PdfSharpCore import-repair.

Clean save & crash reporting

Saving writes a clean, annotation-free snapshot, burns stamps then annotations into a copy via PdfSharpCore XGraphics, and reopens the clean copy so future saves never double-burn. Every save also scrubs the hazards that corrupted files in older builds: dangling /Outlines references, degenerate crop boxes and dead signature values, and the cached PDFium link handle is released first so saving over the open file never hits "being used by another process". A 50-entry timestamped status ring buffer is dumped with structured crash logs to %LOCALAPPDATA%, auto-rotated at a 20 MB cap.

Standards Conformance

New in 1.6.4: every release is validated against veraPDF, the industry reference validator, across a 2,907-file public conformance corpus. The bar is zero regressions.

Every corpus file (the veraPDF test corpus for PDF/A-1/2/4 and PDF/UA-1/2, the Isartor PDF/A-1b suite and the TWG files) is validated pristine, resaved through the normal save pipeline via `--batch-resave`, and validated again. Any file that fails a rule after the resave that it did not fail before counts as a regression. A second pass runs `qpdf --check` on every original / resave pair.

Outcome (v1.6.4, veraPDF 1.30.2)	Files
Corpus total	2,907
Resaved and revalidated	2,236
Refused (encrypted or unparseable; source untouched)	671
Conformance regressions	0 (one documented PDF 2.0 header limitation)
Files that came out more conformant	63
qpdf structural check worsened	0 (195 improved)

Patched at the source

Getting to zero meant fixing the write engine itself. PdfSharpCore is vendored under `third_party/` with six patches, each marked in the source with the ISO clause it satisfies: no Producer/Creator stamping into an imported document's Info dictionary, no `/ModDate` rewrite at open (both break PDF/A's Info-to-XMP equivalence rule, ISO 19005-1 6.7.3), no transparency `/Group` injected onto every page (forbidden in PDF/A-1), stream `/Length` always matching the spec's exact byte count (empty streams included), booleans written as the lowercase `true` / `false` keywords, and the debug verbose file layout removed.

The full report, the comparison script and reproduction instructions ship in the repo under `validation/RESULTS.md`; the summary lives on the technical page at KillerPDF.net.

Notable Engineering Details

A few engineering decisions worth calling out.

- › **Zoom-stable coordinates.** Every annotation lives in the 2048-longest-side render-dim space, identical across all view modes, so marks never drift.
- › **Decoupled bitmap resolution.** Pixels scale up to $\min(6144, 2048 * \text{dpi} * \text{zoom})$ for sharpness while coordinates stay zoom-independent.
- › **Per-tab LRU bitmap cache.** Frozen bitmaps keyed by (page, size-bucket, rotation), evicting whole tabs beyond the three most recent.
- › **Continuous-view virtualization.** Only a window of pages around the viewport holds real bitmaps; released slots keep their exact height so scroll geometry never changes.
- › **Reconcile-in-place tab strip.** A session-to-element cache diffs the strip instead of rebuilding it, so no flicker on switch, drag or resize.
- › **Cross-restart memory.** Fit, zoom, view and page persist per file path (cap 40); lazy tabs avoid rendering every reopened PDF at startup.
- › **A vendored, patched write engine.** PdfSharpCore lives in the repo under `third_party/` with six standards-conformance patches, proven by a 2,907-file veraPDF harness that runs on every release.
- › **Two PdfSharp engines.** PdfSharpCore for the app, PDFsharp 6.2 (separate namespace) only for signing, behind a portable certificate-provider abstraction.
- › **Self-extracting single exe.** Costura embeds managed DLLs; native Tesseract / Leptonica self-extract to a per-version cache and language packs download on demand.

Specifications & Constants

Selected constants and limits, drawn directly from the source.

Specification	Value
Render-dim longest side	2048 DIP (zoom-stable annotation space)
Bitmap resolution cap	6144 px = min(6144, 2048 * dpi * zoom)
Print / OCR render	300 DPI on demand / 2600 px (~300 DPI on Letter)
Zoom range / step	5% to 500%, 15% steps (0.05 / 5.0 / 0.15)
Render-cache tabs (LRU)	3 most-recently-used tabs
Continuous bitmap window	10 pages each side of the viewport, release beyond 15
Per-file state memory	40 documents (DocStates)
Folder / zip import cap	50 files (MaxDropFiles)
Signature /Contents	16,384 bytes, SHA-256, whole chain
Default text annotation	Segoe UI, 14pt
Crash-log status ring	50 entries; logs rotate at 20 MB
Validation corpus	2,907 files (veraPDF + Isartor + TWG suites), zero-regressions bar
UI languages / themes	9 locales / 6 themes plus accent variants

Glossary

Plain-language definitions for the terms used throughout this brochure.

Term	What it means
WPF	Windows Presentation Foundation, Microsoft's native desktop UI framework
partial class	A C# feature that lets one class be written across several files
PDFium	Google's open-source PDF rendering engine that turns a page into pixels
Docnet.Core	The .NET wrapper KillerPDF uses to drive PDFium
PdfPig	A .NET library that extracts text and word positions from a PDF
PdfSharpCore / PDFsharp	Libraries that write PDFs: saves and edits (vendored and patched), and digital signing
vendored	A library's source copied into the repo and built with the app, so it can be patched and audited
veraPDF	The industry reference validator for the PDF/A and PDF/UA standards
PDF/A	The ISO standard for archival PDFs; strict rules so files stay readable for decades
Tesseract	The open-source OCR engine that recognises text in images
DIP	Device-independent pixel, a unit that stays the same apparent size on any display
render-dim	The internal page size (longest side 2048) where annotations are stored
xref	The cross-reference table, a PDF's index of where each object lives
MediaBox	The PDF entry that defines a page's full size
AcroForm	The PDF standard for interactive, fillable form fields
CMS / PKCS#7	The standard cryptographic container that holds a digital signature
OCR / DPI	Optical character recognition / dots per inch (300 DPI is print quality)



Live Showcase

- › A real interactive form to fill and save
- › A printed-style form to mark up by hand
- › Annotation pages seeded for screenshots

Interactive Feedback Form

Real AcroForm fields. Open in KillerPDF (or any reader), fill them in, then Save.

NAME

ROLE

EMAIL

VERSION USED

Which tools do you use most? (tick all that apply)

- | | | | |
|--|---------------------------------------|---------------------------------|----------------------------------|
| ☐ Annotation | ☐ Forms | ☐ OCR | ☐ Signing |
| ☐ Stamp / watermark | ☐ Crop / pages | ☐ Search | ☐ Print |

Overall rating

- | | | | | | | |
|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|------------|--|
| ☐ 1 | ☐ 2 | ☐ 3 | ☐ 4 | ☐ 5 | (5 = best) | ☐ Notify me of releases |
|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|------------|--|

COMMENTS

SIGNATURE

DATE

Tip: use the Signature tool (G) in KillerPDF to sign the line above.

Sample Application Form

A printed-style form with no interactive fields. Ideal for demoing fill-by-annotation, crop and stamp.

KILLERPDF · USER REGISTRATION & EVALUATION

Form KP-1 (Rev. 1.6.4) · Please print clearly in black ink · PAGE 1 OF 1

LAST NAME

FIRST NAME

ORGANISATION

EMPLOYEE ID

EMAIL ADDRESS

PHONE

DATE (YYYY-MM-DD)

LICENSE NO.

EDITION REQUESTED:

☐ Portable (.exe)

☐ Installer

☐ Source (GPL-3)

☐ Enterprise / MSP

PLATFORM:

☐ Windows 10

☐ Windows 11

☐ Server 2019+

☐ Other

COMMENTS

SIGNATURE OF APPLICANT

DATE

Annotation Playground

Seeded content for screenshots. Reach for the toolbar (or the number keys) and mark this page up.

Highlight me (H / 4)

Drag the Highlight tool across this sentence for the carved, theme-tinted band; Strikethrough and Underline give the other two styles.

Draw here (D / 5)

Freehand ink area: round caps, adjustable width, optional eraser. The minimum-length guard ignores accidental taps.



Type a note (T / 2)

Click with the Text tool to drop a typewriter note, then style it from the floating bar.

Draw a line (L / 3)

Use the Line tool with Level snapping for a horizontal rule beneath this row.

Tool	Shortcut	Output type
Highlight	H / 4	Highlight annotation
Draw	D / 5	Ink annotation
Stamp	S / 0	Flattened overlay
Crop	C / 8	CropBox / TrimBox

Annotation Playground II

Seeded content for screenshots. Reach for the toolbar (or the number keys) and mark this page up.

Highlight me (H / 4)

Drag the Highlight tool across this sentence for the carved, theme-tinted band; Strikethrough and Underline give the other two styles.

Draw here (D / 5)

Freehand ink area: round caps, adjustable width, optional eraser. The minimum-length guard ignores accidental taps.



Type a note (T / 2)

Click with the Text tool to drop a typewriter note, then style it from the floating bar.

Draw a line (L / 3)

Use the Line tool with Level snapping for a horizontal rule beneath this row.

Tool	Shortcut	Output type
Highlight	H / 4	Highlight annotation
Draw	D / 5	Ink annotation
Stamp	S / 0	Flattened overlay
Crop	C / 8	CropBox / TrimBox



COLOPHON

Designed and built by Steve the Killer

This brochure was generated for KillerPDF v1.6.4; every figure and constant is drawn directly from the application source, and the conformance numbers come from the validation run shipped with this release.

The wordmark above is set in Typewriter a602 by dead postman (2004), the app's logo face. Body type is Helvetica; technical call-outs are set in Courier.

KillerPDF is free software under the GPL-3.0 license; a source archive is emitted on every release build, and the PDF write engine is vendored and patched in the open under `third_party/`.

KillerPDF.net